

Index page:

```
require('dotenv').config();

const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const compression = require('compression');
const { initSentry, Handlers } = require('./utils/sentry');
const { rateLimiter } = require('./middleware/rateLimiter');
const { errorHandler } = require('./middleware/errorHandler');
const { requestLogger } = require('./middleware/requestLogger');
const logger = require('./utils/logger');
const db = require('./models/db');
const redis = require('./utils/redis');
```

// Routes

```
const authRoutes = require('./routes/auth');
const leadsRoutes = require('./routes/leads');
const automationRoutes = require('./routes/automation');
const webhookRoutes = require('./routes/webhooks');
const healthRoutes = require('./routes/health');
const companiesRoutes = require('./routes/companies');
```

```
const app = express();
```

// Trust reverse proxy (nginx, Railway, Render) — required for Twilio signature verification

```
app.set('trust proxy', 1);
```

// — Sentry (must be first) —————

```
initSentry(app);
app.use(Handlers.requestHandler());
```

// — Security Middleware —

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'"],
      imgSrc: ["'self'", 'data:'],
    },
  },
  hsts: { maxAge: 31536000, includeSubDomains: true, preload: true },
}));
```

```
app.use(cors({
  origin: process.env.ALLOWED_ORIGINS?.split(',') || '*',
  methods: ['GET', 'POST', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-API-Key'],
  credentials: true,
}));
```

// — Core Middleware —

```
app.use(compression());
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));
app.use(requestLogger);
app.use(rateLimiter);
```

// — Routes —

```
app.use('/health', healthRoutes);
app.use('/auth', authRoutes);
app.use('/companies', companiesRoutes);
```

```

app.use('/leads', leadsRoutes);
app.use('/automation', automationRoutes);
app.use('/webhooks', webhookRoutes);

// 404 handler
app.use((req, res) => {
  res.status(404).json({ success: false, error: 'Route not found' });
});

// — Sentry Error Handler —————
app.use(Handlers.errorHandler());

// — Error Handler —————
app.use(errorHandler);

// — Startup —————
// Only start the HTTP server when run directly (node src/index.js).
// When required by tests (require('../src/index')), just export the app
// so supertest can bind its own ephemeral port — no EADDRINUSE.
async function start() {
  try {
    await db.connect();
    logger.info('✅ Database connected');

    await redis.ping();
    logger.info('✅ Redis connected');

    const PORT = process.env.PORT || 3000;
    app.listen(PORT, () => {
      logger.info(`🌀 Server running on port ${PORT} [${process.env.NODE_ENV}]`);
    });
  }
}

```

```

    } catch (err) {
      logger.error(' ✖ Startup failed:', err);
      process.exit(1);
    }
  }

// Graceful shutdown
process.on('SIGTERM', async () => {
  logger.info('SIGTERM received, shutting down gracefully...');
  await db.end();
  await redis.quit();
  process.exit(0);
});

process.on('unhandledRejection', (reason) => {
  logger.error('Unhandled Promise Rejection:', reason);
});

process.on('uncaughtException', (err) => {
  logger.error('Uncaught Exception:', err);
  process.exit(1);
});

// — KEY FIX: only call start() when this file is run directly —————
// require.main === module → true when: node src/index.js
// require.main === module → false when: require('../src/index') from tests
if (require.main === module) {
  start();
}

module.exports = app;

```

Package.json

```
{
  "name": "saas-automation-platform",
  "version": "1.0.0",
  "description": "Multi-tenant SaaS automation platform for service businesses",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "worker": "node src/workers/automationWorker.js",
    "migrate": "node src/models/migrate.js",
    "seed": "node src/models/seed.js",
    "test": "jest --runInBand --forceExit",
    "lint": "eslint src/"
  },
  "dependencies": {
    "@sentry/node": "^8.0.0",
    "bcryptjs": "^2.4.3",
    "bullmq": "^5.70.1",
    "compression": "^1.7.4",
    "cors": "^2.8.5",
    "dotenv": "^16.0.0",
    "express": "^4.18.0",
    "express-rate-limit": "^6.0.0",
    "helmet": "^7.0.0",
    "ioredis": "^5.0.0",
    "joi": "^17.0.0",
    "jsonwebtoken": "^9.0.0",
    "pg": "^8.11.0",
    "twilio": "^4.0.0",
    "uuid": "^9.0.0",
```

```
    "winston": "^3.8.0"
  },
  "devDependencies": {
    "eslint": "^10.0.1",
    "jest": "^25.0.0",
    "nodemon": "^3.0.0",
    "supertest": "^6.0.0"
  },
  "engines": {
    "node": ">=18.0.0"
  }
}
```

Seed.js

```
require('dotenv').config();

const bcrypt = require('bcryptjs');

const db = require('./db');

const logger = require('../utils/logger');

async function seed() {

  logger.info('🌱 Seeding database...');

  const passwordHash = await bcrypt.hash('Password123!', 12);

  await db.transaction(async (client) => {

    // Create demo user

    const { rows: [user] } = await client.query(

      `INSERT INTO users (email, password_hash, first_name, last_name, role)

      VALUES ('demo@example.com', $1, 'Demo', 'User', 'owner')

      ON CONFLICT (email) DO UPDATE SET password_hash = EXCLUDED.password_hash

      RETURNING id`,

      [passwordHash]

    );

    logger.info('Created user:', user.id);

    // Create demo company

    const { rows: [company] } = await client.query(

      `INSERT INTO companies (name, industry, timezone, plan)

      VALUES ('Demo HVAC Co', 'hvac', 'America/New_York', 'growth')

      ON CONFLICT DO NOTHING

      RETURNING id`

    );

    if (company) {
```

```

// Link user to company
await client.query(
  `INSERT INTO company_users (company_id, user_id, role)
  VALUES ($1, $2, 'owner') ON CONFLICT DO NOTHING`,
  [company.id, user.id]
);

// Create automation rules
await client.query(
  `INSERT INTO automation_rules (company_id, name, trigger_type, delay_minutes,
message_template, step_order)
  VALUES
    ($1, 'Immediate Missed Call Follow-up', 'missed_call', 0,
    'Hi {{first_name}}, this is {{company_name}}. We missed your call and want to help! Can we
schedule a time to discuss your HVAC needs?', 1),
    ($1, '30-min Follow-up', 'missed_call', 30,
    'Hi {{first_name}}, just following up from {{company_name}}. We'd love to help with your
HVAC needs. Reply YES to schedule a free estimate!', 2),
    ($1, '24-hour Follow-up', 'missed_call', 1440,
    'Hi {{first_name}}, last message from {{company_name}}. We're still here to help! Book your
free estimate at {{booking_link}}', 3)
  ON CONFLICT DO NOTHING`,
  [company.id]
);

logger.info('Created company and automation rules:', company.id);
}
});

logger.info('✅ Seed complete');
logger.info(`\n📧 Login: demo@example.com / Password123!`);
}

```

```
if (require.main === module) {  
  seed()  
  .then(() => process.exit(0))  
  .catch((err) => {  
    logger.error('Seed failed:', err);  
    process.exit(1);  
  });  
}
```

```
module.exports = { seed };
```

Docker

— Build Stage —————

FROM node:20-alpine AS base

RUN apk add --no-cache dumb-init

WORKDIR /app

Copy package files

COPY package*.json ./

— Development Dependencies —————

FROM base AS dev-deps

RUN npm ci

— Production Dependencies —————

FROM base AS prod-deps

RUN npm ci --omit=dev

— Production Image —————

FROM node:20-alpine AS production

RUN apk add --no-cache dumb-init

WORKDIR /app

Create non-root user

RUN addgroup -g 1001 -S nodejs && \

adduser -S nodeapp -u 1001 -G nodejs

Copy production deps

COPY --from=prod-deps --chown=nodeapp:nodejs /app/node_modules ./node_modules

Copy application code

COPY --chown=nodeapp:nodejs src/ ./src/

COPY --chown=nodeapp:nodejs package*.json ./

Create logs directory

RUN mkdir -p logs && chown nodeapp:nodejs logs

USER nodeapp

Health check

HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \

CMD node -e "require('http').get('http://localhost:3000/health', (r) => r.statusCode === 200 ? process.exit(0) : process.exit(1))"

EXPOSE 3000

Use dumb-init for proper signal handling

ENTRYPOINT ["dumb-init", "--"]

CMD ["node", "src/index.js"]

ci.yml

name: CI/CD Pipeline

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

env:

REGISTRY: ghcr.io

IMAGE_NAME: \${github.repository}

jobs:

— Test —

test:

name: Test

runs-on: ubuntu-latest

services:

postgres:

image: postgres:16-alpine

env:

POSTGRES_DB: saas_automation_test

POSTGRES_USER: postgres

POSTGRES_PASSWORD: testpassword

ports:

- 5432:5432

options: >-

--health-cmd pg_isready

--health-interval 10s

--health-timeout 5s

--health-retries 5

redis:

image: redis:7-alpine

ports:

- 6379:6379

options: >-

--health-cmd "redis-cli ping"

--health-interval 10s

--health-timeout 5s

--health-retries 5

env:

NODE_ENV: test

DB_HOST: localhost

DB_PORT: 5432

DB_NAME: saas_automation_test

DB_USER: postgres

DB_PASSWORD: testpassword

REDIS_HOST: localhost

REDIS_PORT: 6379

JWT_SECRET: test_jwt_secret_not_for_production

ENCRYPTION_KEY: test_encryption_key_32chars_!!

TWILIO_ACCOUNT_SID: ACtest

TWILIO_AUTH_TOKEN: test_token

TWILIO_PHONE_NUMBER: "+15550000000"

PORT: 3001

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Setup Node.js

uses: actions/setup-node@v4

with:

node-version: '20'

cache: 'npm'

- name: Install dependencies

run: npm ci

- name: Run migrations

run: npm run migrate

- name: Run tests

run: npm test -- --coverage

- name: Lint

run: npm run lint

- name: Upload coverage

uses: codecov/codecov-action@v3

if: always()

— Build & Push Docker Image —————

build:

name: Build & Push

runs-on: ubuntu-latest

needs: test

if: github.ref == 'refs/heads/main' || github.ref == 'refs/heads/develop'

permissions:

contents: read

packages: write

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Login to Container Registry

uses: docker/login-action@v3

with:

registry: \${{ env.REGISTRY }}

username: \${{ github.actor }}

password: \${{ secrets.GITHUB_TOKEN }}

- name: Docker meta

id: meta

uses: docker/metadata-action@v5

with:

images: \${{ env.REGISTRY }}/\${{ env.IMAGE_NAME }}

tags: |

type=sha,prefix={{branch}}-

type=ref,event=branch

type=raw,value=latest,enable=\${{ github.ref == 'refs/heads/main' }}

- name: Build and push

uses: docker/build-push-action@v5

with:

context: .

push: true

tags: \${{ steps.meta.outputs.tags }}

labels: \${{ steps.meta.outputs.labels }}

cache-from: type=gha

cache-to: type=gha,mode=max

— Deploy to Production —————

deploy:

name: Deploy

runs-on: ubuntu-latest

needs: build

if: github.ref == 'refs/heads/main'

environment: production

steps:

- name: Deploy to Render

env:

RENDER_API_KEY: \${{ secrets.RENDER_API_KEY }}

RENDER_SERVICE_ID: \${{ secrets.RENDER_SERVICE_ID }}

run: |

curl -X POST \

-H "Authorization: Bearer \$RENDER_API_KEY" \

-H "Content-Type: application/json" \

"https://api.render.com/v1/services/\$RENDER_SERVICE_ID/deloys"

- name: Notify Slack on success

if: success()

uses: slackapi/slack-github-action@v1

with:

payload: |

{

"text": "✅ Production deploy successful: \${{ github.sha }}"

}

env:

SLACK_WEBHOOK_URL: \${{ secrets.SLACK_WEBHOOK_URL }}

- name: Notify Slack on failure

if: failure()

uses: slackapi/slack-github-action@v1

with:

payload: |

{

"text": "❌ Production deploy FAILED: \${{ github.sha }}"

}

env:

SLACK_WEBHOOK_URL: \${{ secrets.SLACK_WEBHOOK_URL }}

Webhooks.test.js

```
// tests/webhooks.test.js

require('dotenv').config();

const request = require('supertest');
const app = require('../src/index');
const db = require('../src/models/db');
const { normalizePhone } = require('../src/utils/phone');

// — Mock the queue so tests never enqueue real BullMQ jobs into Redis —————
// Without this, the missed-call test enqueues a job with a fake company_id,
// and the worker retries it forever with a foreign key error.
jest.mock('../src/services/queue', () => ({
  scheduleWebhookJob: jest.fn().mockResolvedValue({ id: 'mock-job-id' }),
  scheduleAutomationJob: jest.fn().mockResolvedValue({ id: 'mock-job-id' }),
  cancelLeadJobs: jest.fn().mockResolvedValue(0),
  getQueueStats: jest.fn().mockResolvedValue({}),
  automationQueue: { close: jest.fn() },
  resurrectionQueue: { close: jest.fn() },
  webhookProcessingQueue: { close: jest.fn() },
  automationEvents: { close: jest.fn() },
}));

// —————
describe('normalizePhone()', () => {

  it('normalizes 10-digit US number to E.164', () => {
    expect(normalizePhone('5551234567')).toBe('+15551234567');
  });

  it('normalizes 11-digit US number with leading 1', () => {
    expect(normalizePhone('15551234567')).toBe('+15551234567');
```

```
});
```

```
it('keeps already-formatted E.164 number unchanged', () => {  
  expect(normalizePhone('+15551234567')).toBe('+15551234567');  
});
```

```
it('strips formatting characters', () => {  
  expect(normalizePhone('(555) 123-4567')).toBe('+15551234567');  
  expect(normalizePhone('555-123-4567')).toBe('+15551234567');  
  expect(normalizePhone('+1 (555) 123-4567')).toBe('+15551234567');  
});
```

```
it('returns null/undefined as-is', () => {  
  expect(normalizePhone(null)).toBeNull();  
  expect(normalizePhone(undefined)).toBeUndefined();  
});  
});
```

```
// _____
```

```
describe('POST /webhooks/missed-call', () => {
```

```
  let companyId;
```

```
  beforeAll(async () => {  
    const { rows: [c] } = await db.query(  
      `INSERT INTO companies (name, industry, timezone)  
      VALUES ('Webhook Test Co', 'hvac', 'America/New_York') RETURNING id`  
    );  
    companyId = c.id;  
  });
```

```
afterAll(async () => {  
  await db.query('DELETE FROM companies WHERE id = $1', [companyId]);  
});
```

```
it('returns 202 and queues job for valid payload', async () => {  
  const res = await request(app)  
    .post('/webhooks/missed-call')  
    .send({  
      company_id: companyId,  
      caller_phone: '+15551234567',  
      caller_name: 'John Smith',  
    });
```

```
  expect(res.status).toBe(202);  
  expect(res.body.success).toBe(true);  
  expect(res.body.message).toMatch(/queued/i);  
});
```

```
it('returns 404 for unknown company_id', async () => {  
  const res = await request(app)  
    .post('/webhooks/missed-call')  
    .send({  
      company_id: '00000000-0000-0000-0000-000000000000',  
      caller_phone: '+15551234567',  
    });
```

```
  expect(res.status).toBe(404);  
  expect(res.body.success).toBe(false);  
});
```

```
it('returns 400 when caller_phone is missing', async () => {
```

```

const res = await request(app)
  .post('/webhooks/missed-call')
  .send({ company_id: companyId });

expect(res.status).toBe(400);
});

it('returns 400 when company_id is not a valid UUID', async () => {
  const res = await request(app)
    .post('/webhooks/missed-call')
    .send({ company_id: 'not-a-uuid', caller_phone: '+15551234567' });

  expect(res.status).toBe(400);
});
});

// _____

describe('POST /webhooks/twilio-inbound — keyword handling', () => {

  let companyId, leadId;

  beforeAll(async () => {
    const { rows: [c] } = await db.query(
      `INSERT INTO companies (name, industry, timezone)
      VALUES ('Twilio Test Co', 'hvac', 'America/New_York') RETURNING id`
    );
    companyId = c.id;

    const { rows: [l] } = await db.query(
      `INSERT INTO leads (company_id, phone, first_name, status)
      VALUES ($1, '+15559876543', 'Jane', 'contacted') RETURNING id`,

```

```

    [companyId]
  );
  leadId = l.id;
});

afterAll(async () => {
  await db.query('DELETE FROM companies WHERE id = $1', [companyId]);
});

it('handles STOP keyword — marks lead DNC and returns unsubscribe TwiML', async () => {
  const res = await request(app)
    .post('/webhooks/twilio-inbound')
    .type('form')
    .send({ From: '+15559876543', Body: 'STOP', MessageSid: 'SM_test_stop' });

  expect(res.status).toBe(200);
  expect(res.text).toContain('unsubscribed');

  const { rows: [lead] } = await db.query(
    'SELECT status FROM leads WHERE id = $1', [leadId]
  );
  expect(lead.status).toBe('do_not_contact');
});

it('handles START keyword — re-enables lead and returns resubscribe TwiML', async () => {
  const res = await request(app)
    .post('/webhooks/twilio-inbound')
    .type('form')
    .send({ From: '+15559876543', Body: 'START', MessageSid: 'SM_test_start' });

  expect(res.status).toBe(200);

```

```
expect(res.text).toContain('resubscribed');
```

```
const { rows: [lead] } = await db.query(  
  'SELECT status FROM leads WHERE id = $1', [leadId]  
);  
expect(lead.status).toBe('new');  
});
```

```
it('handles HELP keyword — returns help TwiML', async () => {  
  const res = await request(app)  
    .post('/webhooks/twilio-inbound')  
    .type('form')  
    .send({ From: '+15559876543', Body: 'HELP', MessageSid: 'SM_test_help' });
```

```
  expect(res.status).toBe(200);  
  expect(res.text).toContain('STOP');  
});
```

```
it('handles normal reply — stores message and returns empty TwiML', async () => {  
  const res = await request(app)  
    .post('/webhooks/twilio-inbound')  
    .type('form')  
    .send({ From: '+15559876543', Body: 'Yes I am interested', MessageSid: 'SM_test_reply' });
```

```
  expect(res.status).toBe(200);  
  expect(res.text).toBe('<Response></Response>');
```

```
  const { rows } = await db.query(  
    `SELECT * FROM messages WHERE lead_id = $1 AND direction = 'inbound' AND provider_sid =  
    'SM_test_reply',  
    [leadId]
```

```
);  
expect(rows.length).toBe(1);  
expect(rows[0].body).toBe('Yes I am interested');  
});
```

```
it('returns empty TwiML for unknown phone number', async () => {  
  const res = await request(app)  
    .post('/webhooks/twilio-inbound')  
    .type('form')  
    .send({ From: '+15550000000', Body: 'Hello', MessageSid: 'SM_test_unknown' });  
  
  expect(res.status).toBe(200);  
  expect(res.text).toBe('<Response></Response>');  
});  
});
```